

API ETL Project – End-to-End Workflow Documentation

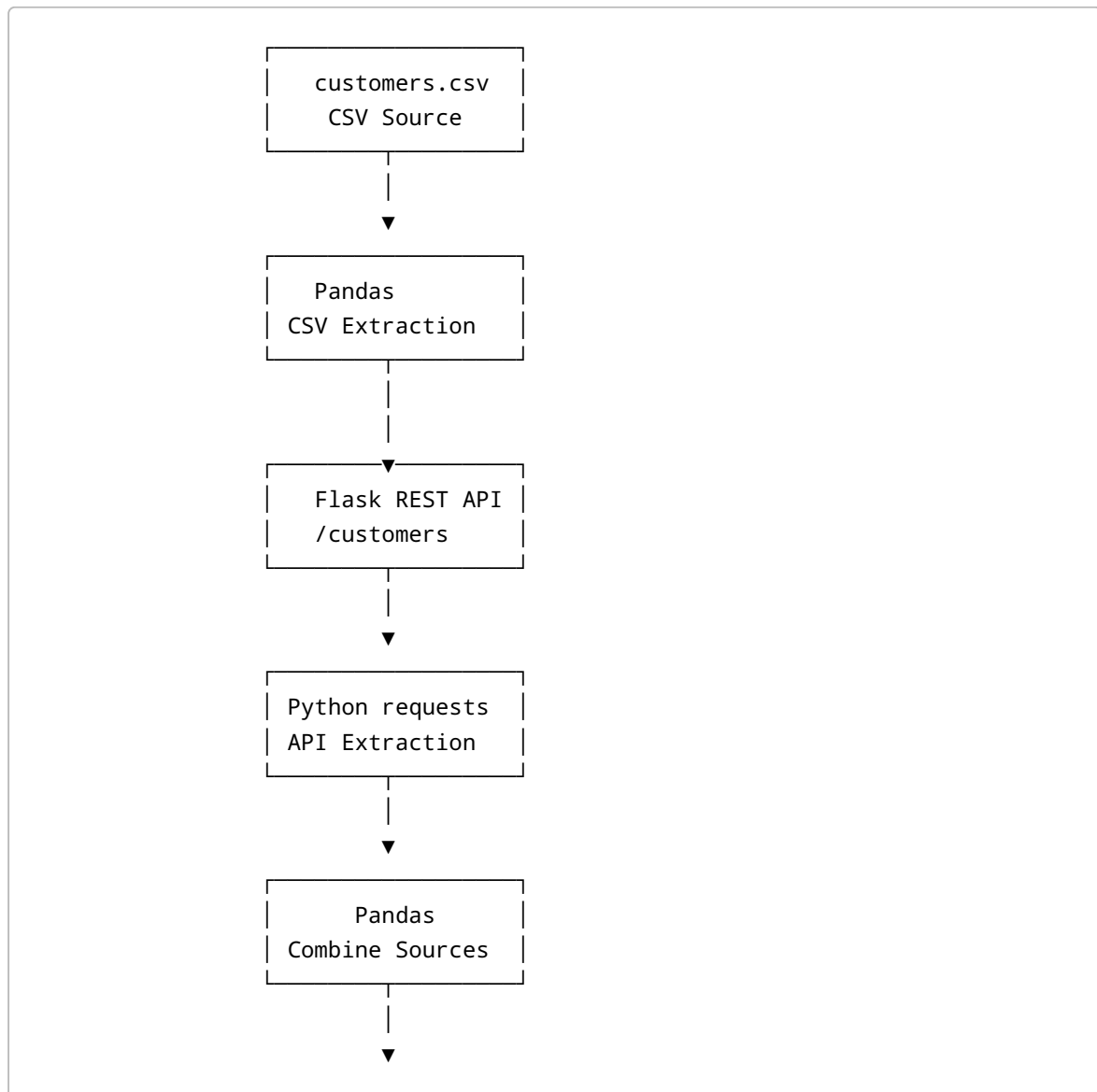
1. Project Overview

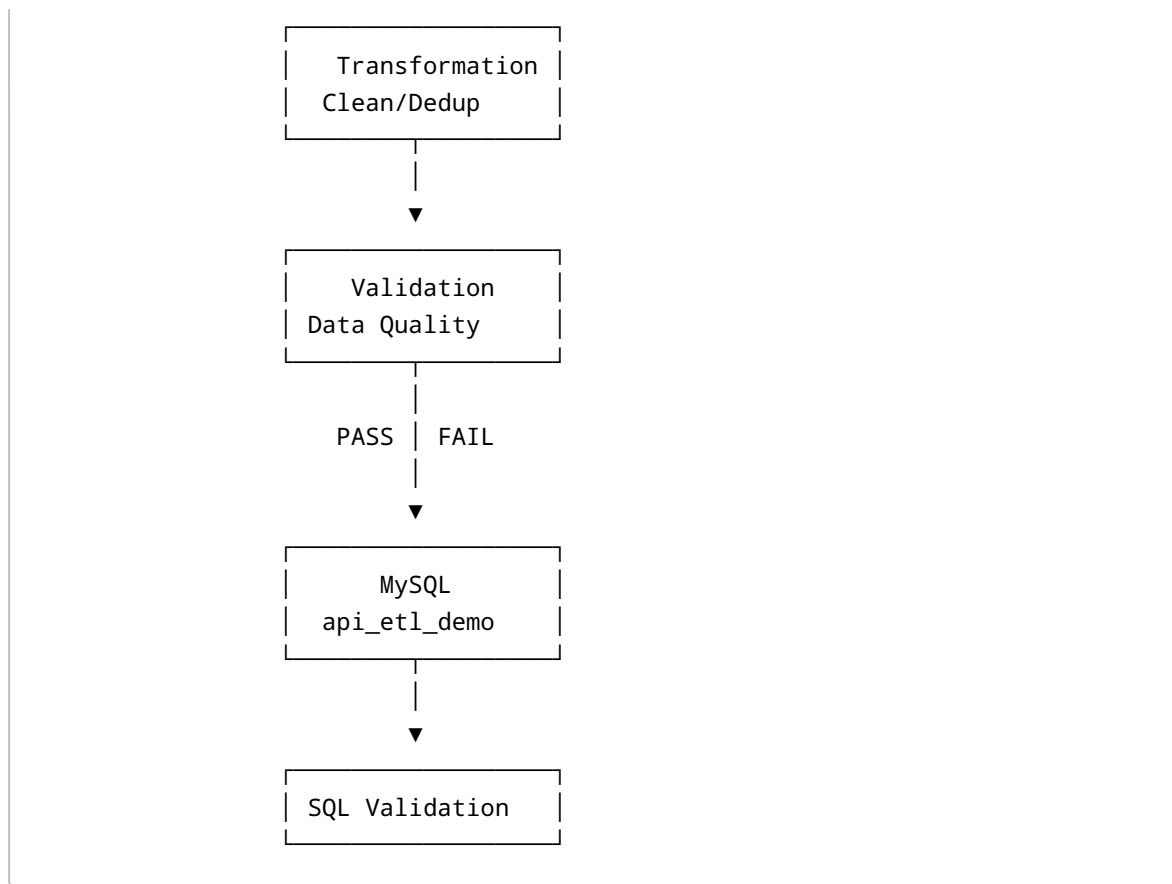
The `api_etl_project` demonstrates a practical ETL pipeline where data comes from **two different sources**:

1. A CSV file
2. A REST API built using Flask

The pipeline combines data from both sources using Pandas, performs transformations and validations, and finally loads the processed data into a MySQL database.

The complete workflow is:





2. Business Scenario

Assume a company receives customer information from two systems.

Source 1 – CSV

The company receives a file:

```
customers.csv
```

containing customer records.

Example:

```
customer_id,customer_name,email,city,state
106,Priya Nair,priya@example.com,Chennai,Tamil Nadu
107,Rahul Verma,rahul@example.com,Delhi,Delhi
```

Source 2 – REST API

Another application exposes customer information through:

```
GET /customers
```

Example response:

```
[
  {
    "customer_id": 101,
    "customer_name": "Ravi Kumar",
    "email": "ravi@example.com",
    "city": "Hyderabad",
    "state": "Telangana"
  }
]
```

The objective is to combine both sources and create one trusted customer dataset in MySQL.

3. Project Structure

```
api_etl_project/
├── app.py
├── data/
│   ├── api_customers.json
│   └── customers.csv
├── etl/
│   ├── __init__.py
│   ├── extract_csv.py
│   ├── extract_api.py
│   ├── transform.py
│   ├── validate.py
│   └── load.py
├── scripts/
│   ├── __init__.py
│   └── run_etl.py
├── sql/
│   ├── 01_create_database.sql
│   └── 02_validation_queries.sql
├── tests/
│   ├── __init__.py
│   └── conftest.py
```

```
|   |─ test_api.py
|   |─ test_validation.py
|   └─ test_etl.py
|
|─ .env.example
|─ .gitignore
|─ requirements.txt
└─ README.md
```

Each component has a specific responsibility.

4. Flask API

The file:

```
app.py
```

implements the REST API.

The API provides:

```
GET    /health
GET    /customers
GET    /customers/<customer_id>
POST   /customers
PUT    /customers/<customer_id>
DELETE /customers/<customer_id>
```

The API uses:

```
data/api_customers.json
```

as its simple training data store.

In a real application, this could instead be backed by:

```
MySQL
PostgreSQL
MongoDB
Cloud Database
Another REST API
```

5. API Health Check

The endpoint:

```
GET /health
```

returns:

```
{  
  "status": "UP"  
}
```

This is useful for checking whether the API is available before starting an ETL process.

Example:

```
curl http://127.0.0.1:5000/health
```

Expected:

```
{  
  "status": "UP"  
}
```

6. Extracting Data from the API

The ETL uses:

```
etl/extract_api.py
```

The Python `requests` library calls the API.

Conceptually:

```
ETL  
|  
▼  
requests.get()  
|  
▼
```

```
Flask API
|
▼
JSON Response
|
▼
Pandas DataFrame
```

The important code is:

```
response = requests.get(
    url,
    timeout=10
)

response.raise_for_status()

payload = response.json()

api_df = pd.DataFrame(payload)
```

7. Why `timeout` Is Important

Consider:

```
requests.get(url)
```

If the API becomes unavailable or stops responding, the ETL could wait indefinitely.

Therefore we use:

```
requests.get(
    url,
    timeout=10
)
```

This means:

Do not wait longer than 10 seconds for the API response.

This is an important production ETL practice.

8. Why `raise_for_status()` Is Important

The API could return:

```
200
```

or:

```
404
500
503
```

Instead of manually checking every status code:

```
if response.status_code != 200:
    ...
```

we use:

```
response.raise_for_status()
```

This causes HTTP errors to raise an exception.

Therefore:

```
API Success
  ↓
Continue ETL

API Failure
  ↓
Exception
  ↓
ETL stops
```

9. CSV Extraction

The file:

```
etl/extract_csv.py
```

handles CSV extraction.

The pipeline uses:

```
pd.read_csv()
```

Example:

```
df = pd.read_csv(  
    "data/customers.csv"  
)
```

The extractor also checks whether required columns exist.

Required columns:

```
customer_id  
customer_name  
email  
city  
state
```

If the CSV contains:

```
customer_id  
customer_name  
email
```

but is missing:

```
city  
state
```

the ETL fails early.

This is called **schema validation**.

10. Two Independent Data Sources

At this point we have:

```
CSV
|
▼
csv_df

API
|
▼
api_df
```

For example:

```
csv_df
-----
106
107
108
109
110
```

and:

```
api_df
-----
101
102
103
104
105
```

There may also be overlapping records.

For example:

```
CSV
103

API
103
```

This creates a potential duplicate.

11. Combining the Sources

The file:

```
etl/transform.py
```

combines the datasets.

The main operation is:

```
combined = pd.concat(  
    [csv_df, api_df],  
    ignore_index=True  
)
```

Conceptually:

CSV
|
├── 106
├── 107
└── 108
|
▼

Pandas
concat

▲
|
API
├── 101
├── 102
└── 103

Result:

```
101  
102  
103  
106
```

```
107
108
```

12. Source Priority

The project implements a simple business rule:

If the same customer exists in both sources, the API record is treated as the latest/
preferred record.

For example:

CSV

```
customer_id = 103
city = Hyderabad
```

API

```
customer_id = 103
city = Secunderabad
```

After transformation:

```
customer_id = 103
city = Secunderabad
```

This demonstrates a common ETL concept:

Source precedence / source priority.

In a real project, this rule would be defined by the business.

13. Data Transformation

The transformation layer performs basic cleansing.

For example:

```
combined_df["email"] = (
    combined_df["email"]
```

```
.str.strip()  
.str.lower()  
)
```

This transforms:

```
"  JOHN@EXAMPLE.COM  "
```

into:

```
"john@example.com"
```

Similarly:

```
combined_df["customer_name"] = (  
    combined_df["customer_name"]  
    .str.strip()  
)
```

removes unwanted leading and trailing spaces.

14. Deduplication

After combining the sources, duplicate customer IDs may exist.

The business key is:

```
customer_id
```

Therefore the ETL performs:

```
drop_duplicates(  
    "customer_id"  
)
```

The objective is:

Before:

```
101  
102  
103
```

```
103
104
```

After :

```
101
102
103
104
```

This prevents duplicate customers from being loaded into the target database.

15. Why Business Keys Matter

The pipeline uses:

```
customer_id
```

as the business key.

This is important because:

```
customer_name
```

is not guaranteed to be unique.

For example:

```
Ravi Kumar
Ravi Kumar
```

could represent two different customers.

Therefore:

```
customer_id
```

is the appropriate identifier for deduplication.

16. Data Validation

After transformation, the pipeline performs validation.

The file:

```
etl/validate.py
```

checks:

```
customer_id → NOT NULL  
customer_id → UNIQUE  
customer_name → NOT NULL  
city → NOT NULL  
state → NOT NULL  
email → valid format
```

The workflow becomes:

```
Extract  
↓  
Transform  
↓  
Validate  
├── PASS → Load  
└── FAIL → Stop
```

This is a very important ETL design principle.

17. Example Validation Failure

Suppose the CSV contains:

```
customer_id = NULL
```

The validation detects:

```
customer_id contains NULL values
```

The ETL produces:

```
VALIDATION FAILED
```

and does not continue to MySQL.

This prevents invalid data from reaching the target.

18. Email Validation

The project also performs basic email validation.

A valid example:

```
john@example.com
```

An invalid example:

```
john-example.com
```

The validation uses a simple regular expression.

This is intentionally basic for training.

Production systems may use:

- Schema validation
 - Pydantic
 - Great Expectations
 - Data quality frameworks
 - Business-specific validation rules
-

19. Loading into MySQL

After validation succeeds:

```
etl/load.py
```

loads the DataFrame into MySQL.

The target database is:

```
api_etl_demo
```

and the target table is:

```
customers
```

The table structure is:

```
CREATE TABLE customers (  
  customer_id INT PRIMARY KEY,  
  customer_name VARCHAR(120) NOT NULL,  
  email VARCHAR(160),  
  city VARCHAR(80) NOT NULL,  
  state VARCHAR(80) NOT NULL  
);
```

20. Pandas → MySQL

The project uses SQLAlchemy and Pandas:

```
df.to_sql(  
    "customers",  
    engine,  
    if_exists="append",  
    index=False  
)
```

The flow is:

```
Pandas DataFrame  
  |  
  ▼  
SQLAlchemy  
  |  
  ▼  
MySQL  
  |  
  ▼  
api_etl_demo.customers
```

21. Why SQLAlchemy?

SQLAlchemy provides the database connectivity layer.

It allows the application to work with:

```
MySQL
PostgreSQL
SQL Server
Oracle
```

with appropriate database drivers and connection URLs.

For this project:

```
SQLAlchemy
+
PyMySQL
+
MySQL
```

are used.

22. Configuration and Secrets

The database connection is stored in:

```
.env
```

Example:

```
TARGET_DB_URL=mysql+pymysql://root:PASSWORD@localhost:3306/api_etl_demo
```

The actual `.env` file should **not** be committed to Git.

Instead the project provides:

```
.env.example
```

with:

```
TARGET_DB_URL=mysql+pymysql://root:YOUR_PASSWORD@localhost:3306/api_etl_demo
```

This allows another developer to create their own `.env`.

23. ETL Orchestration

The file:

```
scripts/run_etl.py
```

acts as the main orchestration layer.

It calls each component in sequence.

The workflow is:

```
run_etl.py
├── extract_csv()
├── extract_api()
├── transform()
├── validate()
└── load_to_mysql()
```

This keeps the individual modules focused on one responsibility.

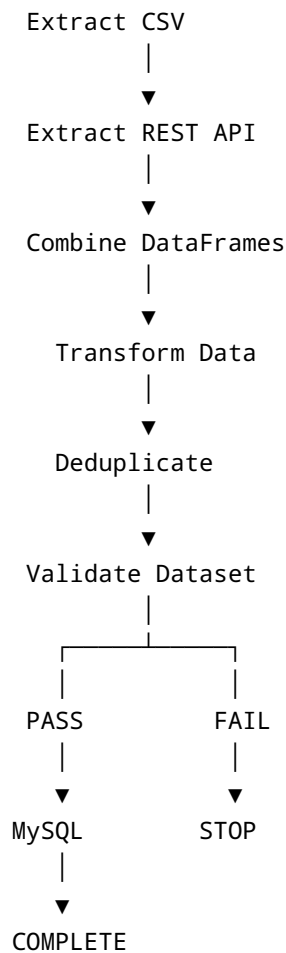
24. Complete ETL Execution

When we run:

```
python3 -m scripts.run_etl
```

the execution is:

```
START
|
▼
```



25. SQL Validation

After the ETL finishes, SQL queries are used to independently validate the target database.

This is important because:

Application-level validation and database-level validation are two different layers.

The SQL validation file is:

```
sql/02_validation_queries.sql
```

26. Row Count Validation

The first validation is:

```
SELECT COUNT(*)  
FROM customers;
```

This answers:

How many customers were loaded?

For example:

```
Expected: 10  
Actual:   10
```

Result:

```
PASS
```

27. Duplicate Validation

The query:

```
SELECT customer_id, COUNT(*)  
FROM customers  
GROUP BY customer_id  
HAVING COUNT(*) > 1;
```

should return:

```
0 rows
```

If it returns:

```
customer_id | count  
-----+-----  
103         | 2
```

then duplicate data exists.

28. NULL Validation

Example:

```
SELECT COUNT(*)  
FROM customers  
WHERE customer_id IS NULL;
```

Expected:

0

Similarly:

```
SELECT COUNT(*)  
FROM customers  
WHERE customer_name IS NULL;
```

should return:

0

29. Business Analysis Queries

The same database can be used for basic analysis.

For example:

```
SELECT city, COUNT(*)  
FROM customers  
GROUP BY city  
ORDER BY COUNT(*) DESC;
```

This produces:

city	customer_count
Hyderabad	3
Delhi	2

Mumbai	1
Chennai	1

This demonstrates how an ETL pipeline eventually enables analytics.

30. Automated Testing

The project includes:

```
tests/
```

Automated tests cover:

```
API
Validation
Transformation
ETL behavior
```

Run:

```
pytest -q
```

31. API Tests

The project tests:

Health endpoint

```
GET /health
```

Expected:

```
200
```

Customer list

```
GET /customers
```

Expected:

200

Existing customer

GET /customers/101

Expected:

200

Missing customer

GET /customers/99999

Expected:

404

32. Validation Tests

The project also tests invalid data.

For example:

```
Duplicate customer_id  
Invalid email  
NULL customer_id
```

This verifies that the data-quality layer itself behaves correctly.

33. Transformation Test

A particularly useful test verifies source precedence.

Example:

CSV:

```
customer_id = 1
customer_name = Old
```

API:

```
customer_id = 1
customer_name = Latest
```

The expected result is:

```
customer_id = 1
customer_name = Latest
```

This ensures the business rule is implemented correctly.

34. API as a Data Source

The project demonstrates:

API → ETL

The API is an upstream source.

```
Flask
|
▼
REST API
|
▼
requests
|
▼
Pandas
```

This is common in real-world data engineering.

Examples of API-based sources include:

```
CRM APIs
Payment APIs
Weather APIs
Marketing APIs
ERP APIs
```

SaaS APIs
Partner APIs

35. API as a Data Sink

The Flask API can also receive data.

For example:

```
curl -X POST
http://127.0.0.1:5000/customers
-H "Content-Type: application/json"
-d '{
  "customer_id": 111,
  "customer_name": "New Customer",
  "email": "new@example.com",
  "city": "Hyderabad",
  "state": "Telangana"
}'
```

The API returns:

201 Created

This demonstrates the reverse direction:

```
ETL
|
▼
POST API
|
▼
Application
```

Therefore an API can act as both:

SOURCE

and:

SINK

depending on the direction of data movement.

36. End-to-End Example

Consider customer 103.

API

```
customer_id = 103  
city = Hyderabad
```

CSV

```
customer_id = 103  
city = Hyderabad
```

The ETL identifies the duplicate.

Only one record is retained.

Now suppose the API changes:

```
customer_id = 103  
city = Bengaluru
```

The next ETL execution prefers the API version.

Final record:

```
103 | Suresh Reddy | ... | Bengaluru | Karnataka
```

This demonstrates:

```
Multiple Sources  
↓  
Source Priority  
↓  
Deduplication  
↓  
Trusted Dataset
```

37. Failure Scenarios

The project intentionally demonstrates several failure points.

API unavailable

```
requests.get()  
  ↓  
Connection Error  
  ↓  
ETL fails
```

API timeout

```
API does not respond  
  ↓  
10-second timeout  
  ↓  
ETL fails
```

Invalid HTTP status

```
500  
  ↓  
raise_for_status()  
  ↓  
ETL fails
```

Invalid CSV schema

```
Missing column  
  ↓  
Schema validation  
  ↓  
ETL fails
```

Duplicate records

```
Duplicate customer_id  
  ↓  
Transformation  
  ↓  
Deduplication
```

Invalid data

```
NULL customer_id
  ↓
Validation
  ↓
FAIL
  ↓
No database load
```

38. Why Validation Happens Before Loading

We don't want this:

```
Bad Data
  ↓
MySQL
  ↓
Dashboard
```

Instead:

```
Raw Data
  ↓
Transform
  ↓
Validate
  ↓
PASS
  ↓
MySQL
```

This is a fundamental ETL principle:

Validate data before it reaches downstream consumers.

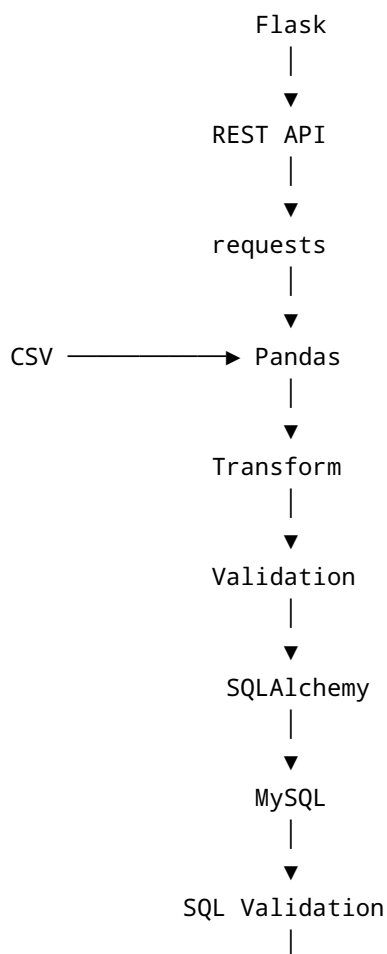
39. Separation of Responsibilities

Each module has one primary responsibility.

Module	Responsibility
<code>app.py</code>	Flask REST API
<code>extract_csv.py</code>	CSV extraction
<code>extract_api.py</code>	REST API extraction
<code>transform.py</code>	Combine/clean/deduplicate
<code>validate.py</code>	Data-quality checks
<code>load.py</code>	MySQL loading
<code>run_etl.py</code>	ETL orchestration
<code>sql/*.sql</code>	Database setup and validation
<code>tests/</code>	Automated testing

This follows the **separation of concerns** principle.

40. Complete Technology Flow



▼
pytest

41. How to Execute the Project

Step 1 – Create environment

```
python3 -m venv venv  
source venv/bin/activate
```

Step 2 – Install dependencies

```
pip install -r requirements.txt
```

Step 3 – Create database

```
mysql -u root -p < sql/01_create_database.sql
```

Step 4 – Configure `.env`

```
cp .env.example .env
```

Update:

```
TARGET_DB_URL=mysql+pymysql://root:PASSWORD@localhost:3306/api_etl_demo
```

Step 5 – Start API

```
python3 app.py
```

Step 6 – Run ETL

Open another terminal:

```
source venv/bin/activate  
python3 -m scripts.run_etl
```

Step 7 – Run tests

```
pytest -q
```

Step 8 – Run SQL validation

```
mysql -u root -p < sql/02_validation_queries.sql
```

42. Recommended Team Explanation

The easiest way to explain this project to the team is:

"We have two upstream sources: a CSV file and a REST API. The Flask application exposes customer data through REST endpoints. Our ETL uses Python requests to consume the API and Pandas to read the CSV. Both datasets are converted into DataFrames and combined. We then apply transformation rules such as trimming strings, normalizing email addresses, applying source priority, and removing duplicate customer IDs. Before loading anything into MySQL, we run data-quality validations. If validation passes, SQLAlchemy and Pandas load the trusted dataset into the MySQL customers table. Finally, SQL queries independently validate row counts, duplicates, NULL values and distributions. Pytest provides automated testing for the API, validation and transformation logic."

43. Key Learning Points for Interns

The intern should understand that an ETL pipeline is not simply:

Read → Write

It is:

```
Extract
↓
Inspect
↓
Transform
↓
Validate
↓
Load
↓
```

Reconcile
↓
Test

The project also demonstrates that APIs can participate in ETL in two directions:

API → ETL

and:

ETL → API

44. Future Production Enhancements

This training project intentionally keeps the architecture simple.

A production version could add:

API

- Authentication
- Authorization
- Pagination
- Rate limiting
- API versioning
- OpenAPI/Swagger
- Pydantic validation

ETL

- Logging
- Retry with exponential backoff
- Batch control
- Incremental processing
- Watermarks
- Dead-letter/error records
- Idempotency

Data Quality

- Great Expectations
- Schema validation
- Data profiling
- Quality metrics
- Quality dashboards

Database

- Transactions
- Connection pooling
- Staging tables
- Audit columns
- Upsert/MERGE logic

Orchestration

Airflow

or:

Databricks Workflows

or:

Azure Data Factory

CI/CD

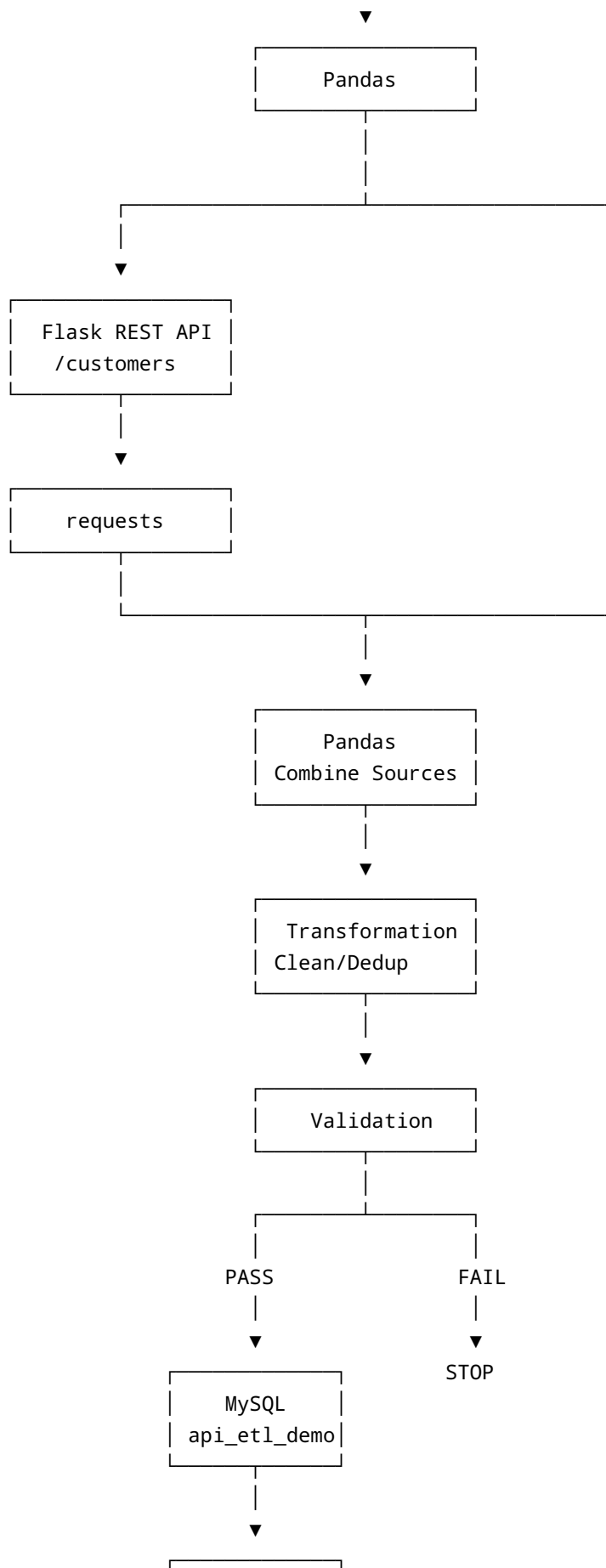
```
Git
↓
GitHub
↓
GitHub Actions
↓
pytest
↓
Build
↓
Deploy
```

45. Final Architecture

The complete project can be summarized as:

```
graph TD; A[CSV Source] --- B[ ];
```

CSV Source





Final takeaway

The main purpose of this project is to demonstrate a **realistic API-driven ETL workflow**:

CSV + REST API → Python/Pandas → Transformation → Data Quality → MySQL → SQL Validation → Automated Tests

It gives interns practical exposure to both **application development (Flask/REST)** and **data engineering (ETL/Pandas/SQL)** in a single project.